



Figure 2: How a request travels

like, what unhealthy looks like, and how confident it is in the diagnosis. The metrics, logs, and traces are the foundation that lets automation make these decisions safely.

Tools like Kubernetes already do a lot of this out-of-the box. Liveness probes restart unresponsive containers. Readiness probes prevent traffic from reaching containers that aren't ready. Horizontal pod autoscalers scale workloads based on metrics like CPU usage or custom application metrics. The basic plumbing for self-healing is increasingly part of the platform itself.

For more sophisticated responses, teams build runbook automation. When alert X fires, run procedure Y. Tools like Rundeck, StackStorm, or even simple webhook-driven systems can handle a lot of routine remediation without waking anyone up.

The best operations teams I've worked with treat their on-call experience as a backlog. Every page in the middle of the night becomes a ticket. The ticket asks: Can we prevent this from happening again, or can we make the response automatic? Over months and years, the noisiest alerts get fixed or automated

away, and the on-call experience becomes genuinely sustainable.

What good dashboards look like

A dashboard is not a list of every metric you have. A dashboard is a story. It should answer specific questions in a clear order.

For most services, I'd start with what the SRE community calls the four golden signals. Traffic, errors, latency, and saturation. Traffic tells you how busy the service is. Errors tell you whether it's failing. Latency tells you how fast it is. Saturation tells you how close to its limits it's running.

If those four signals look healthy, the service is almost certainly fine. If any of them is off, you have a clue about where to look next.

Beyond the golden signals, your dashboard should include the specific things that matter for that service. Queue depth for a queue worker. Cache hit ratio for a cache. Number of active connections for a database. Whatever the specific health indicators are.

What I would not put on a top-level dashboard is everything you can possibly measure. If the dashboard has a hundred panels, nobody will actually look at it during an incident. They'll

get overwhelmed and click away. A good dashboard fits on one screen. It tells you in five seconds whether things are okay. If they're not, it gives you the next place to click.

The hidden cost of telemetry

Let me say something that vendors won't. Observability is not free, even when the tools are open source.

Every metric you collect costs storage. Every log you ship costs bandwidth and storage. Every trace you sample costs both. At a small scale, this is invisible. At a large scale, it can become enormous. I've seen teams whose observability bill rivalled their compute bill. Not because they were doing anything wrong, just because they were collecting everything from everywhere and never going back to ask if they actually used it.

The discipline that helps here is the same discipline that helps with code. Be intentional. Sample wisely. Aggregate where you can. Set retention policies that match how long you actually need the data. Most teams need full-fidelity data for a few days for incident investigation, lower-resolution data for a few months for trend analysis, and almost nothing beyond that.